PROJET DE FIN DE MODULE · AGENT LLM

Concert Agent

Trouver des concerts avec un agent LLM qui utilise de vrais tools.

- Projet Python / Flask
- SQLite
- Agent LLM avec confirmation
- Connexion Spotify et import des artistes
- Démo en 3 scénarios

NOTES ORAL

Présenter le projet en une phrase : « un assistant qui ne se contente pas de répondre, il agit : il cherche, filtre, propose et enregistre — avec validation humaine. »

Le problème

Un chatbot classique peut inventer une date, une salle ou un prix.

Or, pour un concert, une information inventée est inutilisable :

- mauvaise date ;
- mauvaise ville ;
- prix faux ;
- aucune trace de ce qui a été fait.

Objectif : un agent **fiable, traçable et contrôlable**.

NOTES ORAL

Notes : insister sur la différence entre « répondre » et « agir avec des données réelles ».

La solution

Cinq piliers :

1. **Agent LLM** : comprend la demande en français.
2. **Tools Python** : cherchent réellement les concerts.
3. **SQLite** : stocke préférences, favoris, historique et traces.
4. **Confirmation** : aucune écriture sans accord explicite.
5. **Spotify** : importe les artistes réellement écoutés par l'utilisateur.

NOTES ORAL

Notes : l'agent ne fait pas tout lui-même ; il choisit un tool, Python l'exécute, puis le résultat revient au modèle.

Démo 1 : recherche avec préférences

Demande

> Trouve un concert de mes artistes préférés en Auvergne-Rhône-Alpes le mois prochain.

Ce que fait l'agent

- `consulter_preferences` ;
- `rechercher_concerts` avec la région et la période ;
- réponse avec les concerts trouvés par Ticketmaster.

Résultat

- Djadja & Dinaz — Halle Tony Garnier, Lyon ;
- Gims — LDLC Arena, Décines.

Chaque concert affiche : date, heure, lieu, prix et lien de billetterie.

NOTES ORAL

Notes : montrer les traces dans l'interface. Les données viennent de Ticketmaster ; si aucun événement n'est trouvé, la démo est signalée.

Démo 2 : aucun résultat

Demande

> Je veux voir un concert à Paris demain, max 5 €.

Ce que fait l'agent

- recherche réelle avec les filtres ;
- aucun concert ne correspond ;
- réponse : « Je ne peux pas inventer de dates ni de salles. »

Point clé

L'absence de résultat est une **vraie réponse**, pas un échec.

NOTES ORAL

Notes : c'est la preuve que le modèle n'est pas seul ; les données viennent du tool.

Démo 3 : écriture avec confirmation

Demande

> Ajoute le concert de Gims à Décines à mes favoris.

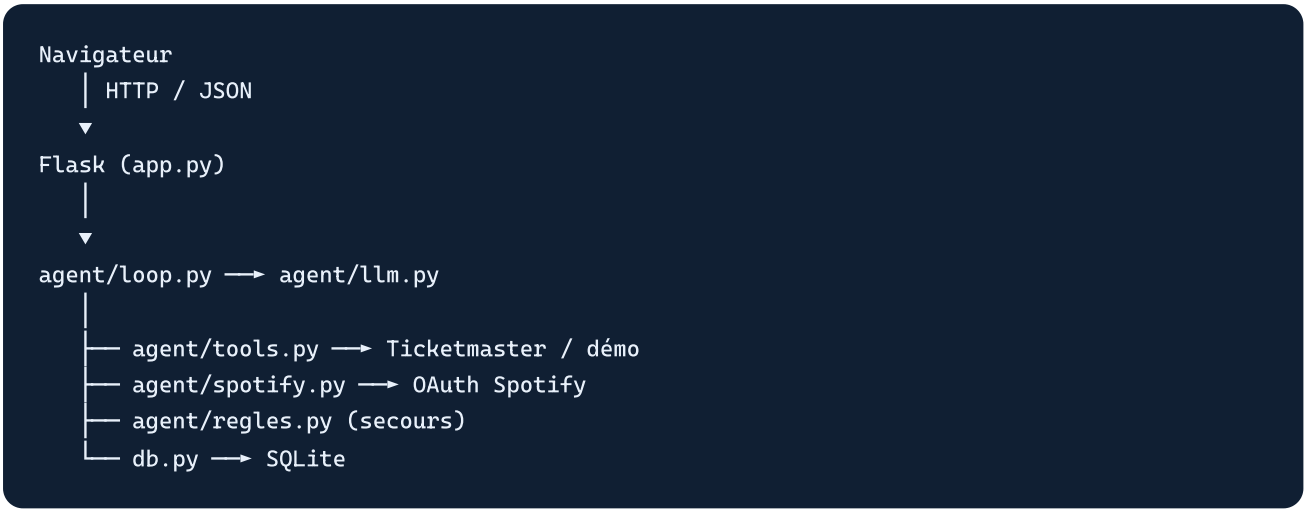
Séquence

1. L'agent prépare l'action.
2. Il demande : « Je vais ajouter Gims — ... à Décines (LDLC Arena). Confirmer ? »
3. L'utilisateur clique sur « Oui ».
4. Le tool `ajouter_favori` écrit dans SQLite.
5. Le favori apparaît dans la barre latérale.

NOTES ORAL

Notes : montrer aussi le bouton « Non » et vérifier qu'aucun favori n'est ajouté.

Architecture



- Une seule porte d'entrée pour le LLM.
- Une seule porte d'entrée pour les tools.
- Une seule base pour les données et les traces.

NOTES ORAL

Notes : expliquer que cette séparation rend le projet testable et remplaçable (Gemini, OpenAI, Ollama).

Les tools

Tool	Lecture / écriture
`consulter_preferences`	lecture
`rechercher_concerts`	lecture
`lister_favoris`	lecture
`historique_agent`	lecture
`ajouter_favori`	écriture
`ajouter_artiste_suivi`	écriture

Chaque tool a une description en français : c'est le mode d'emploi du LLM.

NOTES ORAL

Notes : le LLM ne voit pas le code Python ; il voit le nom, la description et le schéma JSON des arguments.

SQLite et traçabilité

Tables principales :

- `preferences` ;
- `artistes_suivis` ;
- `favoris` ;
- `interactions` ;
- `tool_traces` ;
- `pending_actions` ;
- `resultats_cache` ;
- `spotify_sessions`.

Chaque appel de tool laisse une trace : nom, arguments, résultat, statut, durée.

Chaque favori est cliquable : sa fiche réunit date, heure, salle, ville, prix, source et lien de billetterie.

NOTES ORAL

Notes : montrer `/api/historique` ou le panneau « Traces » dans le chat, puis ouvrir la fiche d'un favori.

Confirmation et sécurité

- Les tools d'écriture sont marqués `is_write=True`.
- L'action est stockée dans `pending_actions`.
- La réponse « Oui » exécute ; « Non » annule.
- `ajouter_favori` vérifie que le concert vient de la dernière recherche.
- Les secrets sont dans `.env`, hors Git.
- Les jetons Spotify sont en base, pas dans le cookie ; le `state` OAuth est vérifié.

NOTES ORAL

Notes : la confirmation est une barrière technique, pas seulement une phrase dans le prompt.

Tests

```
.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Résultat :

```
Ran 13 tests ... OK
```

Les tests couvrent les 3 scénarios, la confirmation, le refus, l'historique, les préférences, les tools et l'intégration Spotify sans appel réseau.

NOTES ORAL

Notes : les tests utilisent une base temporaire et ne dépendent pas du réseau.

Limites et perspectives

Aujourd'hui :

- concerts réels via Ticketmaster (`locale=fr-fr`) ; couverture France partielle ;
- LLM réel validé avec `gemini-3.5-flash-lite` ; mode règles en secours ;
- connexion Spotify réelle validée localement (profil + import) ;
- Spotify limité à 5 comptes en mode développement ;
- données de démo clairement signalées.

Demain :

- élargir la couverture concerts (Bandsintown / OpenAgenda) ;
- déploiement Render avec disque persistant ;
- notifications et partage de favoris.

NOTES ORAL

Notes : annoncer honnêtement ce qui est démontré et ce qui reste à brancher.

Conclusion

Concert Agent montre qu'un LLM peut être utile sans être cru sur parole :

- il choisit ses tools ;
- Python exécute ;
- Spotify fournit les artistes de l'utilisateur ;
- SQLite garde la trace ;
- l'utilisateur confirme les écritures.

NOTES ORAL

Notes : phrase de fin : « L'agent ne devine pas les concerts : il les cherche, et il demande la permission avant d'agir. »